33labs

Flow Audit Report

Introduction

A security review of the Flow protocol was done, focusing on the security aspects of the smart contracts. This audit was performed by 33Labs with Samuel Troy Dominguez and Radoslav Radev as Security Researchers.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any vulnerabilities. Subsequent security reviews, bug bounty programs, and on-chain monitoring are recommended.

About 33 Audits & Company

33Audits is an independent smart contract security researcher company and development group. We conduct audits as a group of independent auditors with various experience and backgrounds. We have conducted over 15 audits with dozens of vulnerabilities found and are experienced in building and auditing smart contracts. We have over 4 years of Smart Contract development with a focus in Solidity, Rust and Move. Check our previous work here or reach out on X @33audits.

About Flow

Flow is a decentralized protocol featuring liquidity management, swap functionality, fee management, factory pattern.

Repository: spotlightcrypto/liquid-launch

Commit: 81dfad10

Scope: spotlightcrypto/liquid-launch/*

Severity Definitions

Severity	Description
Critical	Direct loss of funds, protocol insolvency, or complete system failure
High	Significant loss of funds or major protocol functionality broken
Medium	Moderate impact, partial loss of funds or degraded functionality
Low	Minor impact, limited scope or easily mitigatable

Informational - Findings in this category are recommended changes for improving the structure, usability, and overall effectiveness of the system.

Findings Summary

ID	Title	Severity	Status
[C-01]	User Directly Claiming Through CCA Prevents Users From Actually Receiving Funds	Critical	Fixed
[M-01]	Incorrect Strategy Update Delay For Arbitrum	Medium	Fixed
[M-02]	Strategy update window is bypassed by permissionless migration at `claimBlock`	Medium	Fixed
[M-03]	CCA `AuctionParameters` ABI mismatch can break auction creation when recipients are overridden	Medium	Fixed
[M-04]	Bonus vesting can be griefed by donating tokens to the vault, causing late claims to revert	Medium	Fixed
[M-05]	Allowlist hook validates wrong bidder via AuctionManager	Medium	Fixed
[L-01]	`AuctionManager.sol#setLiquidLaunch()` is missing access control	Low	Fixed
[L-02]	Unlimited Permit2 allowance persists after launch	Low	Fixed
[L-03]	Missing Decimal Validation	Low	Fixed
[L-04]	Incorrect handling of positive ticks in `_createExcessCurrencyPosition`	Low	Fixed
[L-05]	Wrong Calculated Value Returned For `getDeployerClaimable`	Low	Fixed
[I-01]	`LiquidLaunch.sol#launchNewToken()` uses `InvalidReserveAmount()` for totalSupply mismatch	Informational	Fixed
[I-02]	Add validation for `bonusVestDuration` when `bonusAmount > 0` in `LiquidLaunch.sol#_validateLaunchParams()` function	Informational	Fixed
[I-03]	`AuctionManager.sol#transferDeployer()` automatically transfers fee recipient	Informational	Acknowledged
[I-04]	Several event emission inconsistencies/issues	Informational	Fixed

Critical

[C-01] User Directly Claiming Through CCA Prevents Users From Actually Receiving Funds

Description

The protocol attempts to support the scenario where a user (or third party) claims auction tokens directly from Uniswap's (CCA) before interacting with the protocol's `AuctionManager`. This intent is clearly demonstrated by the use of a try/catch around `dist.claimTokens(bidId)`.

However, the implementation incorrectly assumes that `CCA.bids(bidId).tokensFilled` remains readable and accurate after a direct claim has occurred. In reality, CCA destructively consumes this value on first claim, resetting it to `zero`. As a result, the protocol can permanently misaccount users' `tokensFilled`, leading to stuck funds, zero user payouts, and corrupted bonus accounting. Resulting in a user never being able to actually receive funds from their claim, OR opens the possibility of a malicious user to prevent other honest users from receiving funds.

Claim Flow In `AuctionManager`:

1. ALL claim functions call the internal `_claimTokensAndBonus()` function to handle the claim logic:

```
    function _claimTokensAndBonus(address auction, uint256 bidId)
        internal returns (uint128 totalTokensReceived)
    {
        uint128 tokensClaimed;
        BidInfo storage bid = auctionBids[auction][bidId];
        BonusVesting storage vesting = bonusVestings[auction];
        IERC20 token = IERC20(vesting.token);
        IDistributionContract dist = IDistributionContract(auction);

        bid.hasClaimedTokens = true;

  ->      // Read CCA bid state to get tokensFilled (works even if already
claimed by third party)
        IDistributionContract.Bid memory ccaBid = dist.bids(bidId);
  ->      tokensClaimed = ccaBid.tokensFilled.toUint128();

  ->      // Try to claim from CCA - may fail if already claimed by third
party, that's ok
  ->      // The tokens would already be in AuctionManager
  ->      try dist.claimTokens(bidId) {} catch {}

        // Update bid info
  ->      bid.tokensClaimed = tokensClaimed;

        // Calculate and transfer bonus (if configured)
        uint128 bonusReceived = 0;
        uint128 bonusForfeited = 0;
        uint256 curveMultiplierBps = 0;

        if (vesting.totalBonusTokens > 0 && vesting.bonusPeriodStart > 0)
{
            (bonusReceived, bonusForfeited, curveMultiplierBps) =
_calculateAndDistributeBonus(
                auction,
  ->              tokensClaimed
```

```
            );
        }

        // Always track claimed tokens for bonus accounting (even before
vesting starts)
        vesting.totalTokensClaimed += tokensClaimed;

        // Transfer auction tokens to user
        token.safeTransfer(msg.sender, tokensClaimed);
```

2. Notice how the `tokensClaimed` is not a stored value in this protocol, it is read from CCA at the time this function executes

3. Also, the protocol has a try/catch with documentation that it expects to continue to work even if a direct claim in the CCA contract was made. It doesn't revert, instead it will just continue the rest of the function and send tokens to the user that they are entitled to from the claim

What happens when a user first, directly claims through CCA: These are the 2 functions handling the claim in CCA :

```
    function claimTokens(uint256 _bidId) external onlyAfterClaimBlock
ensureEndBlockIsCheckpointed {
        // Tokens cannot be claimed if the auction did not graduate
        if (!_isGraduated()) revert NotGraduated();

        (address owner, uint256 tokensFilled) =
_internalClaimTokens(_bidId);

        if (tokensFilled > 0) {
            Currency.wrap(address(TOKEN)).transfer(owner, tokensFilled);
            emit TokensClaimed(_bidId, owner, tokensFilled);
        }
    }
```

```
   function _internalClaimTokens(uint256 _bidId) internal returns (address
owner, uint256 tokensFilled) {
        Bid storage $bid = _getBid(_bidId);
        if ($bid.exitedBlock == 0) revert BidNotExited();

        // Set return values
        owner = $bid.owner;
        tokensFilled = $bid.tokensFilled;

  ->      // Set the tokens filled to 0
  ->      $bid.tokensFilled = 0;
    }
```

Notice how `$bid.tokensFilled` is RESET to 0

1. So if a user first calls `claimTokens` directly:
    - `tokensFilled` is permanently zeroed
    - No on-chain record of filled tokens remains
    - Subsequent reads of `bids(bidId).tokensFilled` return 0
2. So then, when a user calls `claimTokens` in the `AuctionManager`:

   This current state read of CCA will return the 0 value for `tokensFilled`:

```
      // Read CCA bid state to get tokensFilled (works even if already
claimed by third party)
        IDistributionContract.Bid memory ccaBid = dist.bids(bidId);
       tokensClaimed = ccaBid.tokensFilled.toUint128();
```

3. Which will result in all other logic using 0 as the user's amount of `tokensFilled`, resulting in 0 transfer, 0 bonus calculations:

```
if (vesting.totalBonusTokens > 0 && vesting.bonusPeriodStart > 0) {
            (bonusReceived, bonusForfeited, curveMultiplierBps) =
_calculateAndDistributeBonus(
                auction,
  ->              tokensClaimed
            );
        }

        // Always track claimed tokens for bonus accounting (even before
vesting starts)
 ->       vesting.totalTokensClaimed += tokensClaimed;

        // Transfer auction tokens to user
 ->       token.safeTransfer(msg.sender, tokensClaimed);
```

Impact

1. The protocol anticipates and has some safety mechanisms to still work and allow users to receive their tokens even if a direct call to claim was made on the CCA contracts, BUT the assumption that it would still work, is wrong. It will not work, and user has their funds stuck in the AuctionManager with no way to receive them.
2. As all claim functions call `_claimTokensAndBonus()`:
    - `claimTokens()` → calls `_claimTokensAndBonus()`

    - `claimFullyFilledBid()` → calls `_claimTokensAndBonus()`
    - `claimPartiallyFilledBid()` → calls `_claimTokensAndBonus()`
    - None of the claim functions will work accurately, and this scenario will happen to all of them.

3. **Additional Malicious Attack Path**

    - The `claimTokens` function in the CCA contracts on Uniswap is permissionless (no owner check). A user can call the claim function for any `bidId` and force this scenario upon anyone.
    - The `claimTokens` and `_internalClaimTokens` functions from Uniswap have no ownership validation.
    - A malicious entity can call `claimTokens` as many times and with as many `bidId`s as possible, forcing many users to never receive their funds.

Recommendations

`tokensFilled` has to be cached or stored so that it can be used during the `claim`, even if a third party directly calls CCA and resets `tokensFilled` to 0. The permission-less nature of the claim in Uniswap's CCA makes this difficult as it affects unsuspecting users. But the reliance on the retrieval of the real time storage read of `tokensFilled` from CCA is unreliable and can result in this scenario.

Status

Fixed : The agreed solution was to fork Uniswap's CCA and add access control so that all user actions must route through the `AuctionManager`.

Medium

[M-01] Incorrect Strategy Update Delay For Arbitrum

Description

The `strategyUpdateDelay` parameter for Arbitrum is calculated based on an incorrect block time assumption. The code assumes Arbitrum has ~4 second block times, but according to Arbitrum's official documentation, Arbitrum chains have block times starting at 250ms (0.25 seconds) and can be as low as 100ms. This results in the strategy update delay being approximately **16x shorter than intended** (1.5 hours instead of 1 day), significantly weakening the safety mechanism designed to prevent premature strategy updates.

https://docs.arbitrum.io/how-arbitrum-works/timeboost/gentle-introduction According to Arbitrum's official documentation:

> "In the past, Arbitrum chains ordered incoming transactions on a 'First-Come, First-Serve (FCFS)' basis. This ordering policy is simple to understand and implement, enabling fast block times (starting at 250ms and down to 100ms if desired), and protecting users from harmful types of MEV, such as front-running & sandwich attacks."

The codebase implements the 1 day constraint by calculating the total amount of blocks per day for each respective chain:

```
    /// @notice Strategy update delay in blocks (~1 day)
    /// @dev Set per-chain at deployment:
    ///      - Base: 43200 blocks (2s per block)
    ///      - Polygon: 43200 blocks (2s per block)
 ->   ///      - Arbitrum: 21600 blocks (~4s per block)
    ///      - Ethereum Mainnet: 7200 blocks (12s per block)
    uint256 public immutable strategyUpdateDelay;
```

And that value is set in the constructor upon deployment of LiquidLaunch :

```
    /// @param _liquidLaunchHook The LiquidLaunchHook address for pool
initialization control
  ->  /// @param _strategyUpdateDelay Strategy update delay in blocks (~1
day, chain-specific)
    constructor(
        address _liquidityLauncher,
        address _ccaFactory,
        address _permit2,
        address _auctionManager,
        address _liquidLaunchHook,
        uint256 _strategyUpdateDelay
    ) Ownable(msg.sender) {
        if (_liquidityLauncher == address(0)) revert InvalidAddress();
        if (_ccaFactory == address(0)) revert InvalidAddress();
        if (_permit2 == address(0)) revert InvalidAddress();
        if (_auctionManager == address(0)) revert InvalidAddress();
        if (_liquidLaunchHook == address(0)) revert InvalidAddress();

        liquidityLauncher = _liquidityLauncher;
        ccaFactory = _ccaFactory;
        permit2 = _permit2;
        auctionManager = IAuctionManager(_auctionManager);
        liquidLaunchHook = _liquidLaunchHook;
->        strategyUpdateDelay = _strategyUpdateDelay;
```

According to the `deploy` scripts, the contract deployed on Arbitrum is deployed with `strategyUpdateDelay` based on the assumed 4 second block time. It incorrectly deploys with 21600 blocks, which is for the 4 second / block expectation:

```
 DeployLiquidLaunch.s.sol
    /// @notice Get strategy update delay for the current chain
    /// @dev Returns ~1 day worth of blocks based on chain's block time
    function _getStrategyUpdateDelay() internal view returns (uint256) {
        uint256 chainId = block.chainid;

        if (chainId == BASE_CHAIN_ID || chainId == POLYGON_CHAIN_ID) {
```

```
            return 43200; // 2s blocks
    ->      } else if (chainId == ARBITRUM_CHAIN_ID) {
    ->          return 21600; // ~4s blocks
        } else if (chainId == ETHEREUM_CHAIN_ID) {
            return 7200; // 12s blocks
        } else {
            // Default to Base timing for testnets/unknown chains
            return 43200;
        }
    }
```

Impact

- According to Arbitrum docs, the actual block time is approximately 0.25 seconds / block , not 4 seconds / block.
- Current Implementation:
- Arbitrum delay: **21,600 blocks**
- Assumed block time: **~4 seconds** (as stated in comment)
- Intended delay: 21,600 × 4s = **86,400 seconds = 1 day** ✓
- Actual Arbitrum Block Times:
- Actual block time: **250ms (0.25 seconds)** per Arbitrum documentation
- Actual delay: 21,600 × 0.25s = **5,400 seconds = 1.5 hours**
- So for Arbitrum, the actual delay period as configured currently is `1.5 hours` not the desired 1 day.

Recommendations

Let's use the block time as per Arbitrum documentation, and calculate `strategyUpdateDelay` based on the `0.25s / block` speed.

With that, the actual value for `strategyUpdateDelay` should be :

Using 0.25 seconds per block: Target delay: 86,400 seconds (1 day) Block time: 0.25 seconds per block Blocks needed: 86,400 ÷ 0.25 = 345,600 blocks

Using 0.3 seconds per block (conservative estimate): Target delay: 86,400 seconds (1 day) Block time: 0.3 seconds per block Blocks needed: 86,400 ÷ 0.3 = 288,000 blocks

Status

Fixed

[M-02] Strategy update window is bypassed by permissionless migration at `claimBlock`

Description

`LiquidLaunch.sol#deployLiquidity()` is permissionless and can be called as soon as `block.number >= cca.claimBlock()`.

`LiquidLaunch.sol#updateLiquidityStrategy()` is only allowed after `claimBlock + strategyUpdateDelay`, and it also requires `!launches[auction].liquidityDeployed`.

This creates a race:

- At `claimBlock`, anyone can call `deployLiquidity()` and set `launches[auction].liquidityDeployed = true`.
- After that, `updateLiquidityStrategy()` is permanently blocked (`AlreadyMigrated()`).

So the "escape hatch" is not reliable: if the deployer needs to switch away from a broken strategy, a bot can front-run migration and force the original strategy forever. This broke the intended behavior described in the comments and README.

Recommendations

Make the escape hatch real. Allow `updateLiquidityStrategy()` before migration (i.e., remove the post-claimBlock delay requirement or move it to restrict only permissionless migration).

Status

Fixed

[M-03] CCA `AuctionParameters` ABI mismatch can break auction creation when recipients are overridden

Description

`LiquidLaunch.sol#_overrideAuctionRecipients()` decodes and then re-encodes `auctionConfigData` as `IContinuousClearingAuction.AuctionParameters`, after mutating `fundsRecipient` and `tokensRecipient`.

`IContinuousClearingAuction` interface defines:

- `int24 tickSpacing`
- `uint256 requiredCurrencyRaised`

But canonical/verified Uniswap CCA contracts use different types (commonly `uint256 tickSpacing`, `uint128 requiredCurrencyRaised`). If the deployed factory expects the canonical layout, your re-encoded blob can be rejected or mis-decoded.

If a deployer provides a value for `requiredCurrencyRaised` that is greater than `type(uint128).max`. `LiquidLaunch` smart contract accepts it (uint256), but a canonical factory expecting `uint128` will revert/validate-fail when decoding or checking bounds, causing `launchToken()` to revert during auction creation.

- `LiquidLaunch.sol#_overrideAuctionRecipients()` function do:

```
IContinuousClearingAuction.AuctionParameters memory params =
    abi.decode(auctionConfigData,
(IContinuousClearingAuction.AuctionParameters));
```

```
params.fundsRecipient = vaultAddress;
params.tokensRecipient = vaultAddress;

return abi.encode(params);
```

- `ICCA.sol` interface (struct definition)

```
struct AuctionParameters {
    address currency;
    address tokensRecipient;
    address fundsRecipient;
    uint64 startBlock;
    uint64 endBlock;
    uint64 claimBlock;
    int24 tickSpacing;              // <-- type sensitive
    address validationHook;
    uint256 floorPrice;
    uint256 requiredCurrencyRaised; // <-- type sensitive
    bytes auctionStepsData;
}
```

Recommendation

Make `IContinuousClearingAuction.AuctionParameters` exactly match the deployed CCA's ABI (same field order + types). Do not "approximate" types here since the code re-encode and feed it to the factory.

Recommendations

Make `IContinuousClearingAuction.AuctionParameters` exactly match the deployed CCA's ABI (same field order + types). Do not "approximate" types here since the code re-encode and feed it to the factory.

Status

Fixed

[M-04] Bonus vesting can be griefed by donating tokens to the vault, causing late claims to revert

Description

The bonus vesting math assumes `unsoldTokenAmount` reflects only unsold auction tokens. In reality, `unsoldTokenAmount` is derived from whatever token balance sits in the per-auction `AuctionVault`, because the strategy pulls it via `vault.claimTokens(token)`.

Since anyone can transfer ERC20 tokens to the vault address, an attacker can donate even a small amount of launch tokens to the vault before liquidity deployment. Those donated tokens get swept as part of `claimTokens(token)` and inflate `unsoldTokenAmount`.

`AuctionManager.startBonusVesting()` then reduces `vesting.auctionAmount` by this inflated amount: `vesting.auctionAmount -= unsoldTokenAmount;`

But the actual auction claims (`tokensFilled`) are unaffected by the donation, so `vesting.totalTokensClaimed` will still grow toward the real sold amount. Eventually `vesting.totalTokensClaimed >= vesting.auctionAmount` and the bonus accounting underflows / hits division-by-zero, causing `claimTokens()` to revert before transferring auction tokens. Late claimers get stuck.

- `FullRangeStrategy.sol` - unsold derived from vault token balance:

```
unsoldTokenAmount = vault.claimTokens(token);
```

- `LiquidLaunch.sol` - passes the derived value into AuctionManager:

```
auctionManager.startBonusVesting(auction, config.bonusVestDuration,
uint128(result.unsoldTokenAmount));
```

- `AuctionManager.sol` - shrinks the auctionAmount denominator:

```
vesting.auctionAmount -= unsoldTokenAmount;
```

- `AuctionManager.sol` - bonus math can underflow / break once totalTokensClaimed exceeds auctionAmount:

```
uint128 remainingShares = vesting.auctionAmount -
vesting.totalTokensClaimed;
```

Recommendations

Derive `unsoldTokenAmount` from CCA state (true unsold tokens) instead of `AuctionVault` token balance so external token donations cannot shrink `vesting.auctionAmount` and brick late claims.

Status

Fixed

[M-05] Allowlist hook validates wrong bidder via AuctionManager

Description

Status

Fixed

Low

[L-01] `AuctionManager.sol#setLiquidLaunch()` is missing access control

Description

`AuctionManager.sol#setLiquidLaunch()` is a permissionless one-time setter that can be called by anyone while `liquidLaunch == address(0)`. The

Recommendations

Restrict `setLiquidLaunch()` to `onlyOwner` to prevent accidental or out-of-sequence configuration from being hijacked.

Status

Fixed

[L-02] Unlimited Permit2 allowance persists after launch

Description

`LiquidLaunch.sol#_executeLaunch()` sets an unlimited ERC20 allowance from `LiquidLaunch` to `Permit2` for the launched `token`:

```
IERC20(token).forceApprove(permit2, type(uint256).max);
```

This approval is not scoped to the amount needed for the specific launch and is not reset after the auction is created. Even if `Permit2` is treated as trusted, leaving a permanent unlimited allowance expands blast radius if `Permit2` is compromised or if any unexpected spending path is introduced.

Recommendations

Approve only the exact amount needed for the flow (`auctionAmount` or `totalAmount`), and/or reset the allowance back to 0 after auction creation completes.

Status

Fixed

[L-03] Missing Decimal Validation

Description

The new `LaunchToken` contract that is deployed within the `LaunchNewToken` flow allows the deployer to input the amount of decimals that the newly launched token will have. There are no restrictions on the decimal amount, which is problematic as if the `decimals` value is less than `6`, it will directly violate Uniswap's CCA guidelines.

According to Uniswaps CCA documentation: "Do NOT use the Auction with low-decimal (< 6) tokens. Bidders will lose significant amounts of token due to rounding errors."

```
contract LaunchToken is ERC20, ERC20Permit {
    uint8 private immutable _decimals;

    /// @notice Creates a new LaunchToken with fixed supply
    /// @param name The token name
    /// @param symbol The token symbol
    /// @param totalSupply The total token supply to mint (in base units
with decimals)
    /// @param decimals_ The number of decimals for the token
    /// @param recipient The address to receive the total supply
    constructor(
        string memory name,
        string memory symbol,
        uint256 totalSupply,
        uint8 decimals_,
        address recipient
    ) ERC20(name, symbol) ERC20Permit(name) {
 ->       _decimals = decimals_;
        _mint(recipient, totalSupply);
    }

    /// @notice Returns the number of decimals for the token
    function decimals() public view virtual override returns (uint8) {
        return _decimals;
    }
}
```

Impact

- If a `LaunchNewToken` is created with less than 6 decimals, the actual Launch of the Auction WILL SUCCEED, as uniswap does not revert on that. So the auction will be launched and begin, allowing users to send in `currency` as they place bids.

- But the result will be as clearly stated in the uniswap CCA docs, that users will lose significant funds due to rounding. This puts the users of the auction in danger as they will be subject to significant loss of funds.

Recommendations

Place a restriction on the `decimal` value in either `LaunchNewToken` or the actual `LaunchToken`, preventing a deployer from deploying the new token with less than 6 decimals.

Status

Fixed

[L-04] Incorrect handling of positive ticks in `_createExcessCurrencyPosition`

Description

The `_createExcessCurrencyPosition` function in `FullRangeStrategy.sol` uses incorrect tick rounding logic for positive ticks when calculating the position's upper bound. Unlike the consistent pattern used in clearingTick calculation in `deployLiqudity` and other parts of the codebase, this function applies the negative-tick rounding formula to all ticks, causing positions to be placed one tick spacing further from the current price than intended. In `_createExcessCurrencyPosition`, when `tokenIsCurrency0` is true, the code rounds the current tick DOWN to create an upper bound for the excess currency position. However, it uses the same formula for both positive and negative ticks:

```
alignedTickUpper = currentTick - (config.tickSpacing + remainder);
```

This formula is only correct for negative ticks. For positive ticks, it incorrectly subtracts an extra tickSpacing, placing the position one tick spacing further down than necessary.

1. Comparison with clearingTick Variable Calculation:

Within the `deployLiquidity` function, a local variable clearingTick is calculated to convert the pool's initialized price (from the CCA clearing price) into a tick-aligned value. This calculation correctly handles both positive and negative ticks:

```
// Calculate tick ranges
int24 rawTick = TickMath.getTickAtSqrtPrice(sqrtPriceX96);
// Round tick DOWN (towards negative infinity) to align with tick spacing
int24 clearingTick;
int24 remainder = rawTick % config.tickSpacing;
if (remainder == 0) {
    clearingTick = rawTick;
} else if (rawTick < 0) {
    // For negative ticks: -115136 % 60 = -56, so -115136 - (60 + (-56)) =
-115140
```

```
        clearingTick = rawTick - (config.tickSpacing + remainder);
    } else {
        // For positive ticks: 115136 % 60 = 56, so 115136 - 56 = 115080
        clearingTick = rawTick - remainder;
    }
```

Purpose: This calculation converts the pool's initialized `sqrtPriceX96` (derived from the CCA clearing price) into a tick-aligned value used for positioning liquidity. It correctly checks `rawTick` < 0 and uses different formulas for positive vs negative ticks. This correct pattern is also used in the test files `TickROunding.t.sol` to show the correct handling of rounding for negative vs positive ticks : The test file explicitly shows the correct pattern:

```
function newFloor(int24 rawTick, int24 tickSpacing) internal pure returns
(int24) {
    int24 remainder = rawTick % tickSpacing;
    if (remainder == 0) {
        return rawTick;
    } else if (rawTick < 0) {
        return rawTick - (tickSpacing + remainder);  // Negative ticks
    } else {
        return rawTick - remainder;  // Positive ticks
    }
}
```

2. When Ticks Are Positive

- For a tick to be positive, the pool price must be > 1.0:
- currency/token > 1.0
- This occurs when the currency is worth more per unit than the token

Example:

- Token price: $0.0001 per token
- Currency: $1.00 USDC
- currency/token = 1.00 / 0.0001 = 10,000
- This results in a positive tick

Example 1: Positive Tick:

- INPUT:

```
currentTick = 115136
tickSpacing = 60
remainder = 115136 % 60 = 56
```

- Correct Calculation (as per clearingTick variable calculation in deployLiquidity):

```
alignedTickUpper = 115136 - 56 = 115080
-> excessTickUpper = 115080 - 60 = 115020
Position range: [minTick, 115020]
```

*Current code:

```
alignedTickUpper = 115136 - (60 + 56) = 115136 - 116 = 115020
-> excessTickUpper = 115020 - 60 = 114960
Position range: [minTick, 114960]
```

Difference: The current code places the position at tick `114960` instead of `115020`, which is one tick spacing (60 ticks) further down than intended.

Impact

- For positive ticks, the `_createExcessCurrencyPosition` will alwyas incorrectly calculate the tick at one full tick spacing further than where it actually should be .
- Low impact as the logic is still executed as intended in most cases, but it is accurately incorrect and reduces accuracy of the logic execution

Recommendations

The code should check for negative ticks and use the appropriate formula, handling both cases of positive or negative ticks:

```
int24 remainder = currentTick % config.tickSpacing;
int24 alignedTickUpper;
if (remainder == 0) {
    alignedTickUpper = currentTick;
} else if (currentTick < 0) {
    // For negative ticks: subtract (spacing + remainder)
    alignedTickUpper = currentTick - (config.tickSpacing + remainder);
} else {
    // For positive ticks: subtract just remainder
    alignedTickUpper = currentTick - remainder;
}
```

Status

Fixed

[L-05] Wrong Calculated Value Returned For `getDeployerClaimable`

Description

It was intended that the deployer can retrieve their `immediate` amount, immediately after the launch of an auction, and without waiting for it to graduate or the vesting period to begin. However, the function `getDeployerClaimable` does not handle that properly, and will return `0` for the claimable amount of a deployer, until the actual auction has ended and vesting period has started.

```
    function getDeployerClaimable(address auction) external view returns (
        uint128 immediateClaimable,
        uint128 vestedClaimable,
        uint128 totalClaimed
    ) {
        DeployerVesting storage dv = deployerVestings[auction];
        if (!dv.isInitialized) return (0, 0, 0);

        totalClaimed = dv.claimed;

   ->     // Immediate and vested tokens only claimable after vesting
starts (i.e., after liquidity deployment)
        if (dv.vestingStartTime == 0) {
            return (0, 0, totalClaimed);
        }

        // Calculate immediate claimable
        if (dv.claimed < dv.immediateAmount) {
            immediateClaimable = dv.immediateAmount - dv.claimed;
        }

        // Calculate vested claimable
        uint128 totalVested = _calculateVestedAmount(dv);
        uint128 totalAvailable = dv.immediateAmount + totalVested;

        if (totalAvailable > dv.claimed) {
            vestedClaimable = totalAvailable - dv.claimed -
immediateClaimable;
        }
    }
```

Impact

- The returned value for the actual claimable amount for a deployer will be wrong, as it will return `0` until the auction has ended and the vesting period begins after the deployment of liquidity, even though the deployer is able to claim the `immediate` amount.

Recommendations

Return the `immediate` amount that the deployer can claim, even if the vesting period has not begun.

Status

Fixed

Informational

[I-01] `LiquidLaunch.sol#launchNewToken()` uses `InvalidReserveAmount()` for totalSupply mismatch

Description

If `expectedTotal != totalSupply`, the code reverts with `InvalidReserveAmount()`. That's the wrong error and will confuse integrators and monitoring (it's not a reserve problem; it's a total supply allocation mismatch).

```
if (expectedTotal != totalSupply) {
    revert InvalidReserveAmount();
}
```

Recommendations

Add a dedicated error like `InvalidTotalSupply()` (or reuse a clearer one), and revert with that.

Status

Fixed

[I-02] Add validation for `bonusVestDuration` when `bonusAmount > 0` in `LiquidLaunch.sol#_validateLaunchParams()` function

Description

When `bonusAmount > 0`, there is no validation that `bonusVestDuration > 0`. If `bonusVestDuration = 0` the `_calculateCurveMultiplier()` returns 0 if `bonusDuration == 0`. This means no one can claim any bonus (multiplier is always 0)

Recommendations

Add validation: `if (bonusAmount > 0 && bonusVestDuration == 0) revert InvalidBonusVestDuration();`

Status

Fixed

[I-03] `AuctionManager.sol#transferDeployer()` automatically transfers fee recipient

Description

When transferring deployer role, fee recipient is automatically overwritten to the new deployer. If the original deployer intentionally set a different fee recipient (e.g., a treasury), this gets silently overwritten.

```
    function transferDeployer(address auction, address newDeployer)
external {
        if (msg.sender != auctionDeployers[auction]) revert
Unauthorized();
        if (newDeployer == address(0)) revert InvalidAddress();

        // ... code ...

        // Always transfer fee recipient automatically
        address oldRecipient = feeRecipients[auction];
        feeRecipients[auction] = newDeployer;

        // ... code ...
    }
```

This may cause unintended loss of fee routing configuration.

Recommendations

Consider keeping fee recipient unchanged on deployer transfer or add explicit parameter.

Status

Acknowledged: This is intentional behavior, just as a full reset or in case feeRecipient is compromised

---

[I-04] Several event emission inconsistencies/issues

Description

1) `AuctionVault.claimFailedAuctionFunds()` attributes the claim to the current strategy, not the recipient/caller

On failed auctions, `claimFailedAuctionFunds()` is invoked by `LiquidLaunch` and transfers tokens to the deployer, but the event emits `TokensClaimed(strategy, token, amount)` where `strategy` is the current authorized strategy from `LiquidLaunch`, not the deployer or the caller.

```
address strategy = _getAuthorizedStrategy();
...
```

```
emit TokensClaimed(strategy, token, tokenAmount);
```

2) `AuctionManager.exitBid()` / `exitPartiallyFilledBid()` can emit a refund amount different from what was actually sent

The refund event logs the computed `refundAmount`, but `_sendCurrencyRefund()` caps payouts to the contract's available balance and can send less without reverting. That means the event can overstate what the user actually received.

```
uint256 refundAmount =
CCARefundCalculator.calculateFullyFilledRefund(dist, bidId);
_sendCurrencyRefund(currency, refundAmount);
emit BidExited(auction, msg.sender, bidId, refundAmount.toUint128());
```

The same pattern exists in `exitPartiallyFilledBid()` and in the auto-exit paths (`claimFullyFilledBid`, `claimPartiallyFilledBid`) where `BidExited` is emitted after refund.

Recommendations

1. Emit a dedicated event that includes the real recipient and auction context. Example: `FailedAuctionFundsClaimed(address auction, address deployer, address token, uint256 amount)`.
2. Make `_sendCurrencyRefund()` return the actual amount transferred (`toSend`) and emit that value in `BidExited`.

Status

Fixed

Conclusion

This security audit of LiquidLaunch identified 15 findings (1 Critical, 5 Medium, 5 Low, 4 Informational). The development team addressed 14 of these, with 1 acknowledged. Continued monitoring and periodic security reviews are recommended as the protocol evolves.

This report was prepared by 33Audits & Co and represents our independent security assessment of the LiquidLaunch project.